

OBJECTIVE-PRESERVING, OPTIMIZER-AWARE TASK SAMPLING FOR MULTI-TASK LLM POST-TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-task LLM post-training uses task weights to specify capability priorities and task sampling to allocate compute. Conflating the two makes efficiency claims hard to interpret; global token averaging can also give long-response tasks unintended weight.

We present Gradient-Preserving Adaptive Sampling (GPAS) for multi-teacher on-policy distillation and multi-task reinforcement learning. Each update uses a task-homogeneous batch and gives every response equal weight. GPAS measures task gradients in a metric induced by the optimizer: it recovers raw-gradient sampling for SGD, while our LLM instantiation uses AdamW’s stored second moment. GPAS samples high-impact tasks more often and rescales every sampled gradient by its target weight divided by its sampling probability. The conditional expected gradient remains equal to the user-specified task mixture. The online sampler stores one score per task, and Cost-GPAS also uses measured task time.

In a controlled three-task experiment, GPAS attains a calculated error ratio of 0.698 and a Monte Carlo ratio of 0.697 relative to uniform sampling; the cost-aware proposal lowers its local compute proxy by 10.7%. The completed experiment verifies the estimator calculation. Frozen-gradient diagnostics and three-seed four-teacher MOPD runs will test the AdamW score and progress per token and GPU hour. A smaller three-task GRPO study tests transfer to reward-based updates. The large-model experiments remain pending in this draft.

1 INTRODUCTION

Post-training one model on mathematics, code, instruction following, and science requires two decisions. Target weights state which capabilities the objective values. Sampling probabilities decide which task receives the next rollout, teacher call, and update. Keeping target weights and sampling probabilities separate makes both decisions visible.

Many systems use the sampling ratio as the task weight. Sampling a weak or cheap task more often then changes the model objective, so the resulting capability trade-off can come from the objective, the compute schedule, or both. A coupled design makes optimization efficiency difficult to measure.

Adaptive curricula use task difficulty, learning progress, or domain-loss trajectories to change this mixture online (Corrado et al., 2026; Sun et al., 2026). These methods ask which tasks should receive more weight during training. We study the complementary fixed-target question: can task frequency reduce optimization error and compute while the user-specified capability mixture remains unchanged?

Response length creates a second ambiguity. In a prompt-balanced mixed batch, global token averaging gives long-response tasks a larger coefficient. Recent MOPD work diagnoses this imbalance and changes domain loss weights (Gao et al., 2026). Our training objective gives each response equal weight inside a task-homogeneous batch.

Teacher selection is a prerequisite for OPD. A wide teacher–student gap can create extreme token log-ratios, whereas a narrow gap supplies little signal. Recent studies report stronger transfer at moderate distributional overlap (Ma et al., 2026a; Wang et al., 2026b; Ma et al., 2026b). We train teachers from the student initialization, evaluate them on student rollouts, and study task allocation within the selected teacher set.

Suppose task i has target weight λ_i and sampling probability q_i . Multiplying its sampled gradient by λ_i/q_i keeps the conditional mean equal to the target mixture. The sampling probabilities can then allocate compute while the target mixture stays fixed.

Gradient-Preserving Adaptive Sampling (GPAS) applies classical importance sampling at the task-batch level. GPAS scores task gradients in a metric induced by the training optimizer, then assigns more probability to tasks with larger scores. The identity metric recovers raw-gradient sampling for SGD; for AdamW, we use the lagged preconditioner from its stored second moment. A moving average estimates each task score from completed updates. A probability floor limits stale scores and large importance weights. Cost-GPAS also uses measured task time.

The controlled experiment checks the sampling formulas with known gradient distributions and costs. Frozen LLM gradients compare raw and AdamW-scaled scores without adding training runs. Three-seed MOPD experiments then measure teacher-loss progress per token and GPU hour. A smaller GRPO experiment tests the same task sampler with reward-based updates.

The paper makes three contributions:

- We define a response-normalized task-batch interface that separates a fixed multi-task target from compute allocation in on-policy post-training.
- We derive task allocation in an optimizer-induced metric, recovering raw-gradient sampling for SGD and a lagged-preconditioned rule for AdamW. A cost-aware version also uses measured end-to-end task time, and both samplers estimate their scores online.
- We validate the estimator calculation and specify direct MOPD and GRPO tests that connect score quality to progress per token and GPU hour.

Section 3 defines the objective and sampling interface; Section 4 presents GPAS; Section 5 gives the MOPD and GRPO studies. The appendix contains proofs and implementation details.

2 RELATED WORK

Multi-task optimization. Multi-task methods often learn task weights or modify gradient directions to improve a capability trade-off (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). GPAS starts from task weights supplied by the user. GPAS changes the frequency of task batches and keeps the weighted gradient fixed in expectation. A fixed target makes optimization efficiency measurable under one capability objective.

Multi-task RL for language models. Joint RL post-training can show uneven gradients and uneven task progress (Ramesh et al., 2026; Wu et al., 2025). Difficulty-based curricula sample weak or slowly improving tasks more often (Corrado et al., 2026). Multi-Task GRPO learns robustness-oriented task weights and preserves its realized batch ratios (Ramesh et al., 2026). Our experiments keep the target task weights fixed. Curriculum signals can still define a task proposal, but the sampled gradient receives the same inverse-probability correction as every other proposal.

Multi-teacher on-policy distillation. MOPD routes student rollouts to domain teachers and trains one shared student (Ma et al., 2026a). Related systems use task-specific teacher interfaces and language-specialized teachers (Ma et al., 2026b; Xie et al., 2026). Open-MOPD identifies response length, teacher-student gap, and reused rewards as sources of capability imbalance and changes domain loss weights (Gao et al., 2026). Other OPD studies report unreliable token guidance under large teacher-student distribution mismatch (Wang et al., 2026b;a). Same-origin teachers and moderate next-token overlap improve training stability while preserving a useful learning signal (Ma et al., 2026a;b). Our MOPD protocol uses four same-origin teachers, task-homogeneous fixed-token batches, and one optimizer use per rollout. D³-MOPD adapts the domain mixture from normalized reverse-KL gap and recent descent velocity to exploit asynchronous domain convergence (Sun et al., 2026). In its formulation, the same mixture also weights the MOPD objective. We adapt its trajectory signal as a corrected task proposal, so it can be compared with optimizer-aware sampling under one fixed target. GPAS changes the frequency of complete task batches and applies an inverse sampling factor. Cost-GPAS adds measured GPU time for generation, teacher evaluation, and model updates.

Importance sampling for stochastic optimization. Sampling examples from their gradient size and applying an inverse probability factor can reduce stochastic-gradient variance (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Cost-aware SGD further samples in proportion to gradient scale divided by the square root of evaluation cost and has been applied to GRPO (Mohri et al., 2026). Non-uniform sampling can interact nonlinearly with adaptive optimizers (Lahire, 2023). PiKE adapts data-mixture weights using gradient magnitude, variance, and interactions (Li et al., 2025).

Cost-Aware GRPO decides which already-generated sequences enter policy updates. GPAS decides which task receives the next rollout, teacher call, and optimizer step while preserving a user-specified multi-task objective. GPAS evaluates complete task-batch gradients in an optimizer-induced metric and estimates task scores from completed updates. Its SGD instance recovers classical gradient-norm sampling; its AdamW instance uses a lagged preconditioner. Cost-GPAS applies the published cost-aware sampling identity to the same task-batch setting. Our contribution lies in fixed-target multi-task allocation, response-normalized task batches, and optimizer-aware online scores.

3 TASK OBJECTIVE AND SAMPLING

Task allocation starts with a clear definition of one task gradient. We first average responses inside a task-homogeneous batch. We then sample one task while preserving the user-selected mixture of task gradients.

3.1 DEFINE A COMPARABLE TASK GRADIENT

Consider M tasks. Task i supplies prompts from $\mathcal{D}_i$ and has target weight λ_i , with $\sum_i \lambda_i = 1$. One training draw selects a task and builds a task-homogeneous batch. For response j , let L_j be its number of valid response tokens and let $g_{j,t}$ be its token gradient. The task-batch gradient is

$$G_i = \frac{1}{n} \sum_{j=1}^n \frac{1}{L_j} \sum_{t=1}^{L_j} g_{j,t}. \quad (1)$$

Each response receives equal weight inside its task. A long response supplies more token observations and keeps the same total coefficient as a short response. Every task batch targets a shared number of valid response tokens. Gradient variance can still differ by task, and GPAS measures task-dependent variance. GPU time is recorded separately because prompt length, generation, and teacher evaluation have task-dependent costs.

For MOPD, $g_{j,t}$ is the sampled-token policy gradient. For GRPO, $g_{j,t}$ comes from the clipped policy surrogate. Prefix sampling, token truncation, reward clipping, and gradient clipping are part of the implemented G_i . The sampler preserves the mean of the implemented G_i . Appendix B.1 gives the OPD gradient, and Appendix B.2 describes the GRPO gradient. When the trainer applies task-level clipping, G_i denotes the clipped gradient from Equation 1.

The mean gradient of task i and the target multi-task gradient are

$$h_i = \mathbb{E}[G_i], \quad h = \sum_i \lambda_i h_i. \quad (2)$$

For frozen teachers, λ states the desired distillation mixture.

A full estimate of h requires one complete batch from every task. Sampling one task uses one rollout and one teacher or reward call per optimizer step. The response-normalized G_i provides a common unit for task sampling.

3.2 SEPARATE TASK PRIORITIES FROM SAMPLING FREQUENCY

Let q_i be the probability that task i supplies the next batch. After sampling task I , the trainer sends

$$\hat{g} = \frac{\lambda_I}{q_I} G_I, \quad I \sim q, \quad (3)$$

to the optimizer. When $q_i = \lambda_i$, the multiplier equals one. Other sampling probabilities use the multiplier to keep the target gradient fixed.

Proposition 1 (Conditional objective preservation). *Let $\mathcal{H}_t$ contain the training history before step t , let q_t be determined by $\mathcal{H}_t$ with $q_{t,i} > 0$, and define $h_{t,i} = \mathbb{E}[G_{t,i} \mid \mathcal{H}_t, I_t = i]$. The corrected update preserves the target conditional mean.*

Proof. Because q_t is fixed conditional on $\mathcal{H}_t$, averaging over the task draw and its fresh batch gives

$$\mathbb{E}[\hat{g}_t \mid \mathcal{H}_t] = \sum_i q_{t,i} \frac{\lambda_i}{q_{t,i}} \mathbb{E}[G_{t,i} \mid \mathcal{H}_t, I_t = i] = \sum_i \lambda_i h_{t,i}. \quad (4)$$

□

The weights λ define capability priorities. The probabilities q allocate task batches. Equation 4 assigns a separate role to each quantity. The equality concerns the gradient passed to the optimizer. Stateful optimizers can still produce different parameter paths under different task orders.

An online sampler can use earlier gradients, measured costs, optimizer states, or task-loss trajectories to choose the next q . Every task keeps positive probability, and each new task batch uses fresh rollout randomness, so Proposition 1 applies at every optimizer step. The correction therefore places gradient-based and curriculum-based proposals under the same target; their estimator error and compute cost can be compared directly.

Task allocation now has a measurable objective: choose q so that one corrected task batch estimates h accurately at low cost. Section 4 derives GPAS from gradient scale and Cost-GPAS from gradient scale plus GPU time. The frozen-gradient study tests the task-gradient definition and corrected estimator; the end-to-end MOPD experiment measures training efficiency.

4 GRADIENT-PRESERVING ADAPTIVE SAMPLING

Section 3.2 fixes the target task mixture and leaves the sampling probabilities available for compute allocation. Every expectation in this section conditions on the model, optimizer, and sampler state before the next task is selected.

4.1 SAMPLE TASKS IN THE OPTIMIZER METRIC

Let A_t be a linear map determined by the optimizer state before task selection. It maps gradients into the local update geometry whose estimation error we seek to reduce. Conditional on the training history, A_t is fixed. Suppressing the step index, define

$$\tilde{G}_i = A_t G_i, \quad s_i = \sqrt{\mathbb{E}[\|\tilde{G}_i\|_2^2]}. \quad (5)$$

For SGD, $A_t = I$; its scalar learning rate does not change the proposal. Thus the SGD instance is Raw GPAS. For AdamW, raw gradient norms can be misleading because parameter coordinates have different effective step sizes. Let $\bar{v}_{\text{prev}}$ be AdamW’s stored, bias-corrected second moment before task selection. Its lagged metric is

$$A_t^{\text{AdamW}} G_i = \frac{G_i}{\sqrt{\bar{v}_{\text{prev}}} + \epsilon_{\text{Adam}}}, \quad (6)$$

where division is element-wise. The LLM experiments use this instance and call it GPAS unless stated otherwise.

For the corrected task gradient in Equation 3, the optimizer-metric mean-squared error is

$$\mathbb{E}\|A_t(\hat{g} - h)\|_2^2 = \sum_i \frac{\lambda_i^2 s_i^2}{q_i} - \|A_t h\|_2^2. \quad (7)$$

The part that depends on the proposal is

$$V(q) = \sum_i \frac{\lambda_i^2 s_i^2}{q_i}. \quad (8)$$

Proposition 2 (Optimizer-metric-optimal task proposal). *If $\lambda_i s_i > 0$ for every task, $V(q)$ over the probability simplex is minimized by $q_i^* = \lambda_i s_i / \sum_j \lambda_j s_j$.*

Proof. Cauchy–Schwarz and $\sum_i q_i = 1$ give

$$\left(\sum_i \lambda_i s_i \right)^2 = \left(\sum_i \sqrt{q_i} \frac{\lambda_i s_i}{\sqrt{q_i}} \right)^2 \leq \left(\sum_i q_i \right) V(q) = V(q). \quad (9)$$

Equality requires $q_i \propto \lambda_i s_i$, which proves the claim. $\square$

The identity metric makes the result the classical SGD gradient-norm rule. For AdamW, GPAS uses the stored state because the score must be available before task selection. This lagged score approximates the task-dependent part of the next AdamW step; the frozen-gradient study compares it with the corresponding AdamW task step computed from saved optimizer moments.

4.2 INCLUDE MEASURED TASK COST

Fixed-token task batches can have different runtimes. Let c_i be the measured time for generation, teacher or reward evaluation, actor training, and the optimizer step. Cost-GPAS minimizes the local compute score $J(q) = (\sum_i q_i c_i) V(q)$.

Corollary 1 (Cost-aware task proposal). *If additionally $c_i > 0$, the minimizer of $J(q)$ is $q_i^* = (\lambda_i s_i / \sqrt{c_i}) / \sum_j (\lambda_j s_j / \sqrt{c_j})$.*

Proof. Cauchy–Schwarz gives $J(q) \geq (\sum_i \lambda_i s_i \sqrt{c_i})^2$, with equality exactly when $q_i \propto \lambda_i s_i / \sqrt{c_i}$. $\square$

This applies the cost-aware sampling rule of Mohri et al. (2026) to complete task batches in the selected optimizer metric. $J(q)$ is a local training signal. The frozen-gradient study measures it on held-out task batches, and the end-to-end experiment measures weighted teacher loss against GPU hours.

4.3 ESTIMATE THE SCORES ONLINE

After task i completes, GPAS updates one moving average, $\hat{s}_i^2 \leftarrow \gamma \hat{s}_i^2 + (1 - \gamma) \|\tilde{G}_i\|_2^2$. Cost-GPAS applies the same update to measured task time. A round-robin warm start observes every task. After computing the proposal, the implementation sets probabilities below $q_{\min}$ to the floor and renormalizes the remaining entries. The floor limits score staleness and large importance weights.

Score error changes sampling efficiency smoothly. Appendix B.4 shows that a relative score error of at most δ increases $V(q)$ by at most a factor of $1/(1 - \delta^2)$. The frozen-gradient study evaluates proposals on task batches not used to estimate their scores.

Each completed step records the clipped task gradient before the importance correction. The trainer then multiplies the gradient by λ_I / q_I and passes it to the optimizer. Gradient clipping is applied before the correction so the conditional mean in Equation 4 matches the stated training objective. The experiments use AdamW; changing the optimizer changes the metric, not the correction.

5 EXPERIMENTS

We use three experiments. A controlled Gaussian problem verifies the sampling calculation. Frozen LLM gradients test response normalization and proposal quality without full training runs. Four-teacher MOPD runs measure progress per token and GPU hour, followed by a smaller GRPO transfer study. The controlled experiment is complete; the large-model experiments remain pending.

5.1 CONTROLLED VALIDATION OF THE SAMPLING RULE

We construct three Gaussian task-gradient distributions with different raw gradient scales, AdamW coordinate scales, and task costs. All methods target $\lambda_i = 1/3$ and apply the correction in Equation 3. We draw 20,000 Monte Carlo trials, with 16 corrected task gradients per trial. We omit the probability floor here so that the proposals match the closed-form calculation.

Under the identity metric used by SGD, Raw GPAS is optimal and attains calculated and Monte Carlo error ratios of 0.534 relative to uniform sampling. Figure 1 evaluates the same problem in the AdamW metric. The calculated GPAS error ratio is 0.698, and the Monte Carlo ratio is 0.697. The raw-gradient proposal reaches 7.178 times the observed uniform error because its largest raw coordinates receive a small AdamW scale. Cost-GPAS reaches 0.893 times the uniform time-weighted second-moment proxy. Every corrected proposal has a relative AdamW-scaled mean-gradient error below 2.1%.

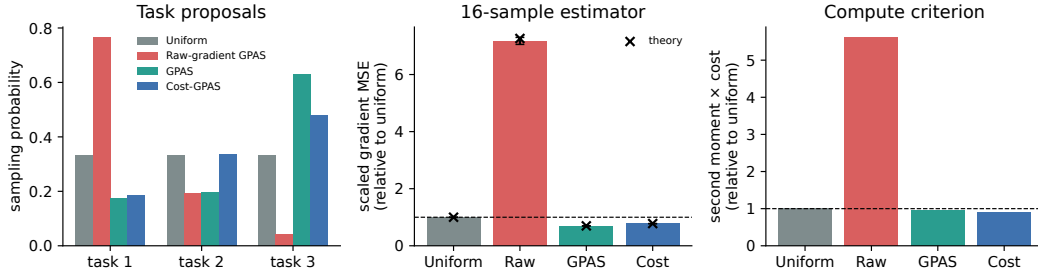


Figure 1: Controlled sampling experiment. Bars show Monte Carlo results normalized by the observed uniform baseline; black crosses show calculated error ratios. Task costs are 5, 2, and 10. Lower values are better in the center and right panels.

5.2 FOUR-TEACHER MOPD SETUP

The MOPD study uses a fixed revision of Qwen3-1.7B (Qwen Team, 2025). Four teachers start from this checkpoint and receive separate post-training on mathematics, code, instruction following, and science question answering. The teachers are frozen during MOPD. We report the model and tokenizer revisions, teacher checkpoints, and each teacher’s score on its training domain. On a shared set of initial-student rollouts, we also report response-normalized reverse KL and the 99th percentile of the absolute teacher–student token log-probability gap.

Each optimizer step samples one task and uses one task-homogeneous batch. Every response is averaged over its valid tokens, responses are then averaged within the task, and one shared task-gradient clip is applied. The trainer records the sampling score before multiplying the gradient by λ_i/q_i and calling AdamW. All methods target $\lambda_i = 1/4$ and use the same data splits.

Each task batch targets 65,536 valid response tokens. The prompt count is estimated from the task’s recent response lengths, and every completed response enters the update. Runs use 600 optimizer steps, an 8,192-token response cap, temperature 1, top- p 1, disabled thinking mode, an eight-step round-robin warm start, $\gamma = 0.95$, and $q_{\min} = 0.05$. We run seeds 42, 43, and 44. Within each seed, methods share the initialization, data order, and evaluation prompts.

5.3 METHODS

Table 1 compares two fixed-target baselines with GPAS and Cost-GPAS. Loss-EMA is a simple difficulty sampler based on the current response-normalized teacher loss. It uses the same moving-average rate and probability floor as GPAS, and the same importance correction keeps its target mixture fixed. Raw GPAS is evaluated only on frozen gradients to isolate the effect of the AdamW metric; it does not require another training run.

Table 1: MOPD methods. Every method uses three training seeds and the same target mixture.

Method	Task proposal	Gradient factor
Uniform	$q_i = 1/4$	1
Loss-EMA	$q_i \propto \lambda_i \widehat{\ell}_i$	λ_i / q_i
GPAS	$q_i \propto \lambda_i \widehat{s}_i$	λ_i / q_i
Cost-GPAS	$q_i \propto \lambda_i \widehat{s}_i / \sqrt{\widehat{c}_i}$	λ_i / q_i

Here $\widehat{\ell}_i$ is the moving average of response-normalized teacher loss, $\widehat{s}_i^2$ tracks $\|A_t G_i\|_2^2$ in the AdamW metric, and $\widehat{c}_i$ tracks total task time. The common floor is applied after each proposal is formed.

5.4 FROZEN-GRADIENT MECHANISM STUDY

We save steps 0, 300, and 600 from the seed-42 Uniform run. At each checkpoint, we collect 16 gradient batches per task. The first eight estimate task scores and costs; the other eight evaluate the resulting proposals.

Response normalization. Using the same cached responses, we compare a prompt-balanced batch with global token averaging against Equation 1 followed by an equal task average. We report response length, the task share induced by global token averaging, and the coefficient after response normalization.

Proposal quality. The held-out half of the gradient bank evaluates Uniform, Loss-EMA, Raw GPAS, GPAS, and Cost-GPAS. We report the AdamW-scaled estimator error $M(q)$ from Equation 13 and the compute score $J(q)$ from Section 4.2, both relative to Uniform. Monte Carlo resampling also compares each corrected mean with the four-task full-mixture mean. This single evaluation tests the importance correction, the benefit of the AdamW metric over raw gradient norms, and the cost-aware proposal.

AdamW score. For every cached task gradient, we compute the task-dependent part of the next AdamW parameter update from the saved first- and second-moment state. Spearman correlations compare the raw gradient norm and the lagged AdamW-metric norm with the resulting update norm.

5.5 END-TO-END MOPD EFFICIENCY

Every 50 steps, evaluation on shared held-out prompts measures weighted teacher loss

$$L(N) = \sum_i \lambda_i \ell_i(N), \quad (10)$$

where N is cumulative valid response tokens or measured GPU hours. We report normalized teacher-loss AUC over the resource range shared by the compared methods and the final weighted and per-task losses. The token comparison is GPAS against Uniform and Loss-EMA; the time comparison is Cost-GPAS against Uniform and GPAS. Results are means and standard deviations over the three training seeds.

The final capability evaluation reports MATH-500 greedy pass@1, a frozen post-cutoff Live-CodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4. The table includes all four task scores and their macro average. Training logs retain the sampled task, proposal probability, importance factor, valid-token count, prompt count, total GPU time, gradient scores, and teacher loss.

5.6 TRANSFER TO MULTI-TASK GRPO

The transfer study uses MATH train, APPS train, and ARC-Challenge train with executable or exact-match rewards (Hendrycks et al., 2021b;a; Clark et al., 2018). Uniform, GPAS, and Cost-GPAS target $\lambda_i = 1/3$ and use the same task-level correction as in MOPD. Each task-homogeneous batch

targets 32,768 response tokens and samples eight responses per prompt. Runs use a six-step round-robin warm start, 400 optimizer steps, AdamW, and seeds 42, 43, and 44.

We report weighted held-out KL-regularized return against tokens and GPU hours, together with final MATH-500, APPS test pass@1, and ARC-Challenge accuracy. This comparison changes only the task sampler and therefore tests whether the same allocation rule transfers from distillation gradients to reward-based updates.

6 DISCUSSION AND CONCLUSION

Multi-task post-training needs separate controls for the target objective and compute allocation. Response-normalized task batches define the unit of the target; inverse sampling preserves its conditional mean. GPAS chooses proposals in an optimizer-induced metric, and Cost-GPAS also uses measured task time. The identity metric recovers the SGD gradient-norm rule; our LLM experiments focus on the lagged AdamW metric.

The controlled experiment supports the local estimator claim. GPAS reduces the measured error by 30.3%, and Cost-GPAS reduces the time-weighted proxy by 10.7%. The constructed comparison shows that coordinate scale can matter. The frozen-gradient study will measure whether the lagged AdamW score tracks the task-dependent AdamW update in LLM training.

Three boundaries guide interpretation. First, the correction preserves the conditional mean of the gradient delivered to the optimizer. Stateful optimizers can still produce different parameter paths under different task orders. Second, the score combines mean scale and batch variation. Frozen-gradient measurements show how this local quantity behaves across tasks. Third, Cost-GPAS optimizes an uncentered local compute score. The GPU-hour curves provide the end-to-end efficiency measurement.

The present draft contains controlled estimator evidence; the MOPD and GRPO results are still pending. The large-model study compares GPAS with uniform sampling and a simple loss-based adaptive sampler while holding the target mixture fixed. The mathematical estimator result and the end-to-end efficiency question remain separate empirical claims.

AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every AI-assisted revision and take responsibility for the final text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with established reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect the reported capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation of the results.

REPRODUCIBILITY STATEMENT

Section 5 specifies the model family, task setup, update budget, evaluation points, and evaluation protocol. The appendix gives the sampling derivations and implementation details. The repository includes the controlled experiment code and generated figure. The large-model release will include data manifests, model and tokenizer revisions, optimizer settings, task-sampling logs, saved model states, and per-prompt evaluation outputs for all three training seeds.

REFERENCES

Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.

- 432 Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradi-
433 ent normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of*
434 *the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
435
- 436 Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and
437 Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning chal-
438 lenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
439
- 440 Nicholas E. Corrado, Wenyan Huang, and Josiah P. Hanna. Distributionally robust multi-task
441 reinforcement learning via adaptive task sampling, 2026. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2605.14350)
442 [2605.14350](https://arxiv.org/abs/2605.14350).
443
- 444 Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He,
445 Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability
446 imbalance in multi-teacher on-policy distillation, 2026. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2608.19098)
447 [2608.19098](https://arxiv.org/abs/2608.19098).
448
- 449 Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin
450 Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge
451 competence with APPS, 2021a. URL <https://arxiv.org/abs/2105.09938>.
452
- 453 Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song,
454 and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In
455 *Advances in Neural Information Processing Systems*, 2021b. URL [https://arxiv.org/](https://arxiv.org/abs/2103.03874)
456 [abs/2103.03874](https://arxiv.org/abs/2103.03874).
457
- 458 Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learn-
459 ing with importance sampling. In *Proceedings of the 35th International Conference on Ma-*
460 *chine Learning*, pp. 2525–2534, 2018. URL [https://proceedings.mlr.press/v80/](https://proceedings.mlr.press/v80/katharopoulos18a.html)
461 [katharopoulos18a.html](https://proceedings.mlr.press/v80/katharopoulos18a.html).
462
- 463 Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023:
464 Optimization for Machine Learning, 2023. URL [https://opt-ml.org/papers/2023/](https://opt-ml.org/papers/2023/paper20.pdf)
465 [paper20.pdf](https://opt-ml.org/papers/2023/paper20.pdf).
466
- 467 Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Pike: Adaptive data
468 mixing for large-scale multi-task learning under low gradient conflicts. In *Advances in Neural*
469 *Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2502.06244>.
470
- 471 Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for
472 multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
473
- 474 Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao,
475 Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy
476 distillation for capability integration in llm post-training, 2026a. URL [https://arxiv.org/](https://arxiv.org/abs/2606.30406)
477 [abs/2606.30406](https://arxiv.org/abs/2606.30406).
478
- 479 Yingzi Ma, Zichen Zhu, Ming Jiang, and Chaowei Xiao. One student, many teachers: Multi-task
480 on-policy distillation via soft-prompt privileged context, 2026b. URL [https://arxiv.org/](https://arxiv.org/abs/2607.18293)
481 [abs/2607.18293](https://arxiv.org/abs/2607.18293).
482
- 483 Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learn-
484 ing, 2026. URL <https://arxiv.org/abs/2604.28020>.
485
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Shyam Sundhar Ramesh, Xiaotong Ji, Matthieu Zimmer, Sangwoong Yoon, Zhiyong Wang,
Haitham Bou Ammar, Aurelien Lucchi, and Ilija Bogunovic. Multi-task grpo: Reliable llm rea-
soning across tasks, 2026. URL <https://arxiv.org/abs/2602.05547>.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL
<https://arxiv.org/abs/1810.04650>.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D³-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL <https://arxiv.org/abs/2608.24987>.
- Chen Wang, Zhaochun Li, Jionghao Bai, Yining Zhang, Hexuan Deng, Ge Lan, and Yue Wang. Distilled reinforcement learning for llm post-training, 2026a. URL <https://arxiv.org/abs/2607.17247>.
- Rui Wang, Hongru Wang, Yi Chen, Boyang Xue, Tianqing Fang, Wenhao Yu, and Kam-Fai Wong. Demystifying on-policy distillation: Roles, pathologies, and regulations, 2026b. URL <https://arxiv.org/abs/2607.13399>.
- Runzhe Wu, Ankur Samanta, Ayush Jain, Scott Fujimoto, Jeongyeol Kwon, Ben Kretzu, Youliang Yu, Kaveh Hassani, Boris Vidolov, and Yonathan Efroni. Imbalanced gradients in rl post-training of multi-task llms, 2025. URL <https://arxiv.org/abs/2510.19178>.
- Yuan Xie, Jiaqi Song, Xianliang Wang, Ming Lei, Jie Gao, and Jie Wu. Language-specialized multi-teacher on-policy distillation for multilingual llm-based asr, 2026. URL <https://arxiv.org/abs/2608.03610>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.

A PRACTICAL TRAINING RECIPE

The recipe uses one complete task batch and one optimizer step per rollout. Task selection occurs before generation, so each step calls one teacher or reward function.

1. Fix the task target. Choose λ from the intended capability priorities and record it before choosing a sampler. For frozen MOPD teachers, λ is the teacher mixture.

2. Define one task gradient. Use a task-homogeneous batch. Average token gradients inside each response and then average responses to form G_i . Apply one shared task-gradient clip. Set a shared valid-token target, and record prompt count, token count, and GPU time.

3. Initialize the sampler. Run a short round-robin warm start. Store a moving average of $\|\tilde{G}_i\|_2^2$ for every task in the selected optimizer metric. This is the raw gradient norm for SGD; the four-task protocol uses AdamW’s stored second moment before the sampling correction. Cost-GPAS also stores a moving average of task time. The protocol uses eight warm-start steps and $\gamma = 0.95$.

4. Sample and optimize. GPAS sets $q_i \propto \lambda_i \hat{s}_i$. Cost-GPAS sets $q_i \propto \lambda_i \hat{s}_i / \sqrt{\hat{c}_i}$. Apply the probability floor, sample one task, compute and clip G_i , record its score, multiply by λ_I / q_I , and call the optimizer. The four-task protocol uses AdamW and $q_{\min} = 0.05$.

5. Save training state. Checkpoint the score and cost averages with the model and optimizer. Log I , q_I , λ_I / q_I , valid tokens, prompt count, total GPU time, and the raw and optimizer-metric gradient norms. These fields are sufficient to resume a run and analyze the sampler.

B DERIVATIONS AND IMPLEMENTATION DETAILS

B.1 SAMPLED-TOKEN OPD

Fix a student-generated prefix z . Write $p_v = \pi_\theta(v \mid z)$ and $\tau_v = T_i(v \mid z)$. The reverse-KL ascent gradient at prefix z is

$$u_i(z) = \sum_v p_v \log \frac{\tau_v}{p_v} \nabla_\theta \log p_v. \quad (11)$$

For a token V sampled from p , the sampled-token gradient is

$$U_i(z, V) = \text{sg} \left[\log \frac{\tau_V}{p_V} \right] \nabla_\theta \log p_V, \quad \mathbb{E}[U_i(z, V) \mid z] = u_i(z). \quad (12)$$

Response normalization averages the prefix gradients according to Equation 1.

B.2 TASK-HOMOGENEOUS GRPO GRADIENT

For task i , GRPO samples response groups from one prompt, computes within-prompt advantages, and applies the standard clipped policy surrogate (Shao et al., 2024). The implementation first averages token terms inside each response and then averages responses inside the task batch. The resulting clipped task gradient is G_i . The GRPO policy ratio handles behavior-policy change inside a task, while λ_i/q_i handles task sampling.

B.3 FROZEN-GRADIENT EVALUATION METRIC

For an evaluation bank, let $\tilde{h} = A_t h$ be the full-mixture mean in the selected optimizer metric, and let $\hat{\tilde{g}} = A_t \hat{g}$ be the transformed corrected draw. The centered error is

$$M(q) = \mathbb{E} \left\| \hat{\tilde{g}} - \tilde{h} \right\|_2^2. \quad (13)$$

The experiment estimates h and the expectation from held-out task batches. Calibration batches supply q , so proposal construction and evaluation use separate samples. The LLM evaluation uses the AdamW metric in Equation 6; the controlled experiment also reports the SGD identity metric.

B.4 APPROXIMATE-SCORE PENALTY

Let $q_i^* = \lambda_i s_i / Z$, where $Z = \sum_i \lambda_i s_i$, and let $\hat{q}_i = \lambda_i \hat{s}_i / \hat{Z}$. Write $w_i = \lambda_i s_i / Z$ and $r_i = \hat{s}_i / s_i$. Then

$$\frac{V(\hat{q})}{V(q^*)} = \left(\sum_i w_i r_i \right) \left(\sum_i \frac{w_i}{r_i} \right). \quad (14)$$

If $r_i \in [1 - \delta, 1 + \delta]$, the Kantorovich inequality gives

$$\frac{V(\hat{q})}{V(q^*)} \leq \frac{((1 - \delta) + (1 + \delta))^2}{4(1 - \delta)(1 + \delta)} = \frac{1}{1 - \delta^2}. \quad (15)$$

The result concerns proposal quality for fixed scores. Model drift, observation noise, and the probability floor are measured separately.

B.5 PROBABILITY FLOOR AND SAMPLER STATE

The online sampler fixes entries below $q_{\min}$ and renormalizes the remaining active entries. Each checkpoint stores the score average, cost average, observation count, and current proposal for every task together with the model and optimizer state.